

Edge cases (context limit) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Story — `context_limit_practice.py` : Doc03 had you guess how many turns fit in a context window — this is where that guess meets a real, deliberately-triggered `BadRequestError` . Seeing the exact exception type and message once means you recognize it instantly later, instead of panicking at an unfamiliar crash. **If not:** the Build Task's "handle a too-long conversation gracefully" requirement would ask you to catch an error you'd never actually seen fire.

All examples below assume `client = OpenAI()` with `.env` already loaded.

Basic Version

Approach 1 — send it, catch whatever happens

```
# context_limit_practice.py
huge_input = "word " * 200_000

try:
    client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": huge_input}],
    )
except Exception as e:
    print("Error type:", type(e).__name__)
    print("Message:", str(e)[:200])
```

Expected output:

```
Error type: BadRequestError
Message: Error code: 400 - {'error': {'message': "This model's maximum
context length is 128000 tokens..."}}
```

This works and shows the real error. Catching the broad `Exception` finds it, but doesn't document which specific class your code is actually expecting — see Intermediate.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — catch the specific error type

```
# context_limit_practice.py
import openai

# why: 200,000 repeated words reliably blows past any model's context
# window, on purpose, so the real error actually fires.
huge_input = "word " * 200_000

try:
    client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": huge_input}],
    )
except openai.BadRequestError as e:
    # how: catching the specific class documents exactly what failure this
    # code expects — a bare except Exception would also hide a real bug.
    print(f"Error type: {type(e).__name__}")
    print(f"Message: {str(e)[:200]}")
```

Expected output:

```
Error type: BadRequestError
Message: Error code: 400 - {'error': {'message': "This model's maximum
context length is 128000 tokens..."}}
```

Difference from Basic: catching `openai.BadRequestError` specifically, instead of a bare `Exception`, documents exactly what failure this code expects and is prepared for — the same "catch the specific error you know how to handle" rule from Doc01. A bare `except Exception` here would also silently swallow a real bug elsewhere in how the request was built, not just this one expected, known failure.

Which one should you actually write? Intermediate's `except openai.BadRequestError` is what this document's Build Task uses — catch it, then trim the oldest turns and keep going, which is a reactive fix that costs one wasted network round-trip per occurrence. A proactive local check with `tiktoken` (counting tokens before sending, so an over-budget request never leaves your machine)

is worth reaching for once you're shipping this for real and want to avoid that wasted round-trip — not required for this exercise or the Build Task.